

FinSight

Smart Personal Finance Management Web Application

SOFTWARE REQUIREMENTS SPECIFICATION

(Prepared in accordance with IEEE Std 830-1998 guidelines)

Document Version: 1.0

Technology Stack: HTML5 · CSS3 · JavaScript (ES6)

Prepared for: GLA University, Greater Noida

Prepared by: Saniya Jaiswal (NC)

Date: 18-08-2026

Table of Contents

1. Introduction

5

1.1 Purpose

5

1.2 Scope

5

1.3 Intended Audience

5

1.4 Definitions, Acronyms, and Abbreviations

6

1.5 References

6

1.6 Document Overview

6

2. Overall Description

7

2.1 Product Perspective

7

2.2 Product Functions (Summary)

7

2.3 User Classes and Characteristics

7

2.4 Operating Environment

7

2.5 Design and Implementation Constraints

8

2.6 Assumptions and Dependencies

8

3. System Features / Functional Requirements

9

3.1 Feature: User Profile & Local Session

9

3.2 Feature: Income & Expense Transaction Management

9

3.3 Feature: Categorization of Transactions

10

3.4 Feature: Financial Dashboard

10

3.5 Feature: Budget Planning & Alerts

10

3.6 Feature: Search, Filtering & Reports

11

3.7 Feature: Data Persistence

11

4. External Interface Requirements

12

4.1 User Interfaces

12

4.2 Hardware Interfaces	12
4.3 Software Interfaces	12
4.4 Communication Interfaces	12
5. Non-Functional Requirements	13
5.1 Performance Requirements	13
5.2 Usability	13
5.3 Reliability & Availability	13
5.4 Security Requirements	13
5.5 Maintainability	13
5.6 Portability	13
5.7 Compliance	14
6. System Architecture & Data Design	15
6.1 High-Level Architecture	15
6.2 Module Breakdown	16
6.3 Data Design (Current Phase — localStorage Schema)	16
6.4 Future Database & API Design (Planned Next Phase)	17
7. Use Case Model	18
7.1 Use Case Diagram (Textual Representation)	18
7.2 Use Case: Add a Transaction	18
7.3 Use Case: Track Budget	18
7.4 Use Case: View Dashboard	19
8. Software Testing Approach	20
8.1 Testing Levels	20
8.2 Sample Test Cases	20
8.3 Acceptance Criteria	20
9. Other Requirements	21

9.1 Assumptions and Dependencies 21

9.2 Future Scope 21

9.3 Risks and Mitigations 21

9.4 Document Approval 21

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete and detailed description of the **FinSight** web application. It defines the functional and non-functional requirements, system constraints, and design considerations necessary to guide the development, testing, and evaluation of the system. This document is intended for use by the student development team, project guide/faculty evaluators, and any reviewer assessing the technical scope and quality of the project.

1.2 Scope

FinSight is a browser-based personal finance management application built using **HTML5, CSS3, and JavaScript (ES6)**. It enables an individual user to record income and expense transactions, categorize spending, view a real-time financial dashboard, and monitor overall financial health through summaries and simple visual charts. The initial version operates as a client-side single-user application using the browser's **Local Storage** for data persistence, with the architecture designed to allow a future upgrade to a server-based system with a database and REST API without a major redesign.

The system is **not** intended to replace professional accounting or tax software, does not process real banking transactions, and does not handle real monetary transfers. It is an academic-grade personal budgeting and expense-tracking tool.

1.3 Intended Audience

- **Students / Developers** — to understand what to build and how modules interact.
- **Project Guide / Faculty Evaluators** — to assess completeness, feasibility, and quality of requirements.
- **Testers** — to design test cases against defined functional requirements.
- **Future Contributors** — to extend the system (e.g., add a backend/database) using the documented architecture.

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
DOM	Document Object Model — the JavaScript interface to HTML
localStorage	Browser-based key-value storage used for client-side data persistence
CRUD	Create, Read, Update, Delete — basic data operations
API	Application Programming Interface
Transaction	A single recorded income or expense entry
Category	A label used to classify a transaction (e.g., Food, Rent, Salary)
Budget	A user-defined spending limit for a category or time period
Responsive Design	Layout that adapts to different screen sizes (desktop, tablet, mobile)

1.5 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- MDN Web Docs — HTML, CSS, and JavaScript language references
- W3C Web Content Accessibility Guidelines (WCAG) 2.1
- ECMAScript 2015+ (ES6) Language Specification

1.6 Document Overview

Section 2 describes the overall product perspective, functions, user classes, and design constraints. Section 3 lists detailed functional requirements grouped by feature/module. Section 4 covers external interface requirements. Section 5 defines non-functional requirements. Section 6 presents the system architecture and data design. Section 7 documents use cases. Section 8 outlines the testing approach, and Section 9 covers assumptions, dependencies, and future scope.

2. Overall Description

2.1 Product Perspective

FinSight is a new, standalone, self-contained web application. It is not a modification of an existing system and does not currently depend on external financial institutions, payment gateways, or third-party banking APIs. It runs entirely within a modern web browser and stores data locally on the user's device for the academic version of the project. The architecture is modular, separating the presentation layer (HTML/CSS), the application/business logic layer (JavaScript modules), and the data layer (Storage Manager abstraction), so that the storage layer can later be swapped for a real backend (database + REST API) with minimal changes to the UI and logic layers.

2.2 Product Functions (Summary)

- Add, edit, and delete income and expense transactions.
- Automatically categorize transactions into predefined and custom categories.
- Calculate and display real-time account balance, total income, and total expenses.
- Present an interactive dashboard with summary cards and visual charts (pie/bar/line).
- Filter and search transactions by date range, category, or transaction type.
- Set monthly budgets per category and alert the user when limits are approached/exceeded.
- Generate simple financial reports/summaries (e.g., monthly spending breakdown).
- Persist all data locally between sessions using browser storage.
- Provide a responsive UI that works across desktop, tablet, and mobile screen sizes.
- Allow data export (e.g., CSV/JSON) and reset/clear-data functionality.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Primary User (Individual)	A single end user who logs personal income and expenses to track their own finances.	Basic computer/browser literacy
Administrator (optional, future scope)	Manages multiple user accounts in a future multi-user version.	Moderate
Evaluator / Faculty	Reviews the system for functional correctness and design quality.	Technical

2.4 Operating Environment

- **Client-side:** Any modern web browser — Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari (latest two major versions).
- **Operating System:** Platform-independent (Windows, macOS, Linux, Android, iOS) since the app runs in-browser.
- **Server-side (current phase):** None required — fully client-side static application.

- **Server-side (future phase):** Node.js/Express or similar backend with a relational or NoSQL database (see Section 6.4).

2.5 Design and Implementation Constraints

- Must be implemented using only HTML5, CSS3, and vanilla JavaScript (ES6) for the core academic deliverable, without mandatory external frameworks.
- Data persistence in the current phase is limited to browser localStorage; data is device- and browser-specific and is not synced across devices.
- No real banking, payment, or financial institution integration is included, for security and academic-scope reasons.
- Single-user context assumed for the current phase — no authentication against a remote server (a simple local login/profile-name screen may be used for personalization only).
- The application must remain lightweight and functional without a persistent internet connection once loaded (basic offline usability).

2.6 Assumptions and Dependencies

- The user's browser supports HTML5, CSS3, ES6 JavaScript, and the Web Storage API.
- The user does not clear browser storage/cache, or is aware this will remove locally stored data.
- Currency handling is limited to a single currency (configurable) for the current phase.
- The evaluation environment provides a standard desktop or laptop browser for demonstration.

3. System Features / Functional Requirements

Each feature below is described with its purpose, detailed functional requirements (uniquely identified as **FR-x.y**), and priority. Priority levels: **High** (core/must-have for MVP), **Medium** (important, should-have), **Low** (optional/stretch goal).

3.1 Feature: User Profile & Local Session

Allows the user to set up a simple local profile so the dashboard can be personalized, without requiring a remote authentication server in the current phase.

ID	Requirement	Priority
FR-1.1	The system shall prompt a first-time user to enter a display name and default currency.	High
FR-1.2	The system shall store the user profile locally and load it automatically on return visits.	High
FR-1.3	The system shall allow the user to edit profile details (name, currency, monthly budget goal) at any time.	Medium
FR-1.4	The system shall provide a "Reset Application Data" option that clears all stored data after confirmation.	Medium

3.2 Feature: Income & Expense Transaction Management

The core feature enabling users to record and manage all financial transactions.

ID	Requirement	Priority
FR-2.1	The system shall allow the user to add a transaction with: amount, type (income/expense), category, date, and optional note.	High
FR-2.2	The system shall validate that the amount is a positive number and that mandatory fields are filled before saving.	High
FR-2.3	The system shall display all transactions in a sortable, scrollable list/table, most recent first by default.	High
FR-2.4	The system shall allow the user to edit an existing transaction's details.	High
FR-2.5	The system shall allow the user to delete a transaction, with a confirmation prompt.	High
FR-2.6	The system shall update the balance and all dashboard summaries immediately after any add, edit, or delete operation.	High

3.3 Feature: Categorization of Transactions

ID	Requirement	Priority
FR-3.1	The system shall provide a predefined set of categories (e.g., Salary, Food, Rent, Transport, Utilities, Entertainment, Shopping, Health, Others).	High
FR-3.2	The system shall allow the user to create, rename, and delete custom categories.	Medium
FR-3.3	The system shall assign a distinct color/icon to each category for visual identification in charts and lists.	Medium
FR-3.4	The system shall prevent deletion of a category that is currently in use, or shall reassign affected transactions to "Others".	Medium

3.4 Feature: Financial Dashboard

A single-page overview giving the user an at-a-glance understanding of their financial status.

ID	Requirement	Priority
FR-4.1	The system shall display summary cards for Total Balance, Total Income, and Total Expenses for the selected period.	High
FR-4.2	The system shall display a pie/donut chart of expense distribution by category.	High
FR-4.3	The system shall display a bar or line chart of income vs. expenses over time (e.g., last 6 months).	Medium
FR-4.4	The system shall allow the user to switch the dashboard view between Daily, Weekly, Monthly, and Yearly periods.	Medium
FR-4.5	The system shall display a short list of the most recent transactions on the dashboard.	Medium
FR-4.6	The dashboard shall update dynamically without requiring a full page reload when data changes.	High

3.5 Feature: Budget Planning & Alerts

ID	Requirement	Priority
FR-5.1	The system shall allow the user to set a monthly budget limit for each expense category.	Medium
FR-5.2	The system shall display a progress indicator (e.g., progress bar) showing spending against the set budget per category.	Medium
FR-5.3	The system shall visually flag (e.g., color change / warning icon) any category where spending exceeds 80% of its budget.	Medium
FR-5.4	The system shall notify the user (in-app banner/alert) when a category's spending exceeds 100% of its budget.	Low

3.6 Feature: Search, Filtering & Reports

ID	Requirement	Priority
FR-6.1	The system shall allow filtering of transactions by type (income/expense), category, and date range.	High
FR-6.2	The system shall provide a text search over transaction notes/descriptions.	Medium
FR-6.3	The system shall generate a monthly summary report showing totals per category.	Medium
FR-6.4	The system shall allow the user to export transaction data as a CSV or JSON file.	Low

3.7 Feature: Data Persistence

ID	Requirement	Priority
FR-7.1	The system shall persist all transactions, categories, budgets, and profile data in browser localStorage.	High
FR-7.2	The system shall automatically reload persisted data when the application is reopened.	High
FR-7.3	The system shall handle storage errors (e.g., quota exceeded) gracefully with a user-facing message.	Low

4. External Interface Requirements

4.1 User Interfaces

- A clean, card-based **Dashboard** screen showing summary statistics and charts.
- A **Transactions** screen with an add/edit form and a filterable, sortable list/table.
- A **Categories** management screen for adding/editing/removing categories.
- A **Budgets** screen for setting and monitoring category-wise budgets.
- A **Reports** screen showing monthly/period summaries and export options.
- A **Settings/Profile** screen for currency, name, and data-reset controls.
- A responsive navigation bar/sidebar accessible on desktop and collapsible on mobile.
- Consistent blue-accented color theme, readable typography, and clear iconography throughout.

4.2 Hardware Interfaces

No special hardware is required. The application runs on any device (desktop, laptop, tablet, or smartphone) capable of running a modern web browser with a display, keyboard, and/or touch input.

4.3 Software Interfaces

Interface	Description
Web Browser	Renders HTML/CSS/JS; provides the Web Storage API (localStorage) used for persistence.
Charting Library (e.g., Chart.js)	Optional client-side library used to render pie, bar, and line charts on the dashboard.
Future: REST API	In a future phase, JSON-based REST endpoints for transactions, categories, and budgets, replacing localStorage calls.
Future: Database	A relational (e.g., MySQL/PostgreSQL) or NoSQL (e.g., MongoDB) database for multi-device, multi-user persistence.

4.4 Communication Interfaces

The current phase requires no network communication beyond loading the static application assets (HTML, CSS, JS, and any CDN-hosted charting library) over HTTP/HTTPS. In the future phase, the client shall communicate with the backend server over HTTPS using JSON payloads via RESTful API calls (see Section 6.4).

5. Non-Functional Requirements

5.1 Performance Requirements

- The dashboard shall reflect a newly added/edited/deleted transaction within 1 second (in-memory update, no server round-trip).
- The application shall load its initial interface within 3 seconds on a standard broadband connection.
- The system shall comfortably support at least 2,000 transaction records in localStorage without noticeable UI lag.

5.2 Usability

- The UI shall follow a consistent blue-based visual theme with clear typography and spacing for readability.
- All primary actions (Add Transaction, View Dashboard) shall be reachable within 2 clicks/taps from any screen.
- Form validation errors shall be displayed inline, in plain language, near the relevant field.
- The interface shall be usable by first-time users without requiring a manual, aided by placeholder text and tooltips.

5.3 Reliability & Availability

- The system shall not lose previously saved data due to normal application use (excluding manual browser cache clearing).
- As a static client-side application, availability equals browser/device availability — no server downtime risk in the current phase.

5.4 Security Requirements

- All input fields shall be validated and sanitized to prevent script injection into the DOM (basic XSS mitigation).
- No sensitive real-world financial credentials (bank logins, card numbers) shall ever be collected or stored by the system.
- In the future server-based phase, passwords shall be hashed (e.g., bcrypt) and all API traffic shall use HTTPS.

5.5 Maintainability

- JavaScript code shall be organized into logical modules (e.g., transactions.js, budgets.js, dashboard.js, storage.js) to ease future maintenance.
- Naming conventions and inline comments shall be used consistently to support handover and future contributions.
- The storage layer shall be abstracted behind a single module so it can be replaced with API calls with minimal changes elsewhere.

5.6 Portability

- The application shall function correctly on the latest two versions of Chrome, Firefox, and Edge, without browser-specific code paths.
- The responsive layout shall adapt correctly to common breakpoints: mobile (<576px), tablet (576–992px), and desktop (>992px).

5.7 Compliance

The UI shall aim to follow WCAG 2.1 Level A/AA guidance where practical (sufficient color contrast, keyboard navigability of forms, descriptive labels) to support basic accessibility as a good academic practice, though full compliance is not mandatory for this project's scope.

6. System Architecture & Data Design

6.1 High-Level Architecture

FinSight follows a layered, client-side **MVC-inspired architecture** to keep concerns separated and the codebase maintainable, even without a formal framework:

Layer	Responsibility	Technology
Presentation Layer	Renders UI screens (Dashboard, Transactions, Budgets, Reports) and captures user input.	HTML5, CSS3
Application/Logic Layer	Handles business rules: calculations, validation, categorization, budget checks, chart data preparation.	JavaScript (ES6 modules)
Data Access Layer	Abstracts read/write operations for transactions, categories, budgets, and profile.	storage.js (wraps localStorage / future API)
Persistence Layer	Stores data across sessions.	Browser localStorage (current) → Database via API (future)

6.1.1 Architecture Diagram (Textual Representation)

```
[ User ] → [ Presentation Layer: HTML/CSS Views ]
      ↓ events (click, submit)
[ Application Logic Layer: JS Modules – transactions.js, budgets.js,
  dashboard.js, categories.js ]
      ↓ read/write calls
[ Data Access Layer: storage.js ]
      ↓
[ Persistence: Browser localStorage ] ■ (future) ■ [ REST API ] → [ Database ]
```

6.2 Module Breakdown

Module (File)	Purpose
index.html / *.html	Page structure for Dashboard, Transactions, Budgets, Reports, Settings.
style.css	Global styling, blue theme variables, responsive layout (Flexbox/Grid, media queries).
app.js	Application entry point; initializes modules and navigation/routing between screens.
transactions.js	Add/edit/delete/list transactions; validation logic.
categories.js	Manage predefined and custom categories.
budgets.js	Set budgets, compute spend-vs-budget, trigger alerts.
dashboard.js	Aggregate data and render summary cards and charts.
storage.js	Central data access layer wrapping localStorage (get/set/remove/clear).
utils.js	Shared helpers: currency formatting, date formatting, ID generation.

6.3 Data Design (Current Phase — localStorage Schema)

Data is stored as JSON-serialized objects under dedicated localStorage keys. The conceptual structure below is designed to map directly onto future database tables.

6.3.1 Transaction Object

Field	Type	Description
id	String (UUID)	Unique identifier for the transaction.
type	String	"income" or "expense".
amount	Number	Transaction amount (positive value).
categoryId	String	Reference to a Category object's id.
date	String (ISO Date)	Date the transaction occurred.
note	String	Optional description/note.
createdAt	String (ISO DateTime)	Timestamp when the record was created.

6.3.2 Category Object

Field	Type	Description
id	String (UUID)	Unique identifier for the category.
name	String	Category display name (e.g., "Food").
type	String	"income" or "expense" category.
color	String (Hex)	Color used for charts/badges.
isDefault	Boolean	True for system-defined categories.

6.3.3 Budget Object

Field	Type	Description
id	String (UUID)	Unique identifier for the budget entry.
categoryId	String	Reference to a Category object's id.
monthlyLimit	Number	Maximum planned spend for the category, per month.
period	String	Target period, e.g., "2026-08".

6.3.4 User Profile Object

Field	Type	Description
name	String	Display name of the user.
currency	String	Currency code/symbol, e.g., "INR", "USD".
monthlyIncomeGoal	Number	Optional target monthly income/savings goal.

6.4 Future Database & API Design (Planned Next Phase)

To evolve FinSight into a multi-device, multi-user system, the following relational schema and REST API are proposed as a natural extension of the current data model:

Table	Key Columns
users	user_id (PK), name, email, password_hash, currency, created_at
categories	category_id (PK), user_id (FK), name, type, color, is_default
transactions	transaction_id (PK), user_id (FK), category_id (FK), type, amount, date, note, created_at
budgets	budget_id (PK), user_id (FK), category_id (FK), monthly_limit, period

Endpoint	Method	Purpose
/api/auth/register, /api/auth/login	POST	User registration and authentication.
/api/transactions	GET / POST	List all transactions / create a new transaction.
/api/transactions/{id}	PUT / DELETE	Update or delete a specific transaction.
/api/categories	GET / POST	List / create categories.
/api/budgets	GET / POST / PUT	Manage category budgets.
/api/reports/summary	GET	Retrieve aggregated summary data for a given period.

7. Use Case Model

7.1 Use Case Diagram (Textual Representation)

```
( Add Transaction )
( Edit / Delete Transaction )
[ User ] ■■ ( View Dashboard )
( Manage Categories )
( Set / Track Budget )
( Filter / Search Transactions )
( Export / Reset Data )
```

7.2 Use Case: Add a Transaction

Field	Description
Use Case ID	UC-01
Actor	User
Precondition	User has the application open on the Transactions screen.
Main Flow	1. User clicks "Add Transaction". 2. User selects type (income/expense). 3. User enters amount, category, date, and optional note. 4. User clicks "Save". 5. System validates input and stores the transaction. 6. System updates balance, dashboard, and transaction list.
Alternate Flow	If validation fails (e.g., empty amount), the system displays an inline error and does not save.
Postcondition	New transaction is persisted and reflected across the application.

7.3 Use Case: Track Budget

Field	Description
Use Case ID	UC-02
Actor	User
Precondition	At least one expense category exists.
Main Flow	1. User navigates to the Budgets screen. 2. User sets a monthly limit for a category. 3. System saves the budget. 4. As expenses are logged in that category, system recalculates spend-vs-budget and updates the progress indicator. 5. System flags/alerts when thresholds are crossed.
Postcondition	User can see real-time budget utilization for each category.

7.4 Use Case: View Dashboard

Field	Description
Use Case ID	UC-03
Actor	User
Precondition	Application has loaded, with or without existing transaction data.
Main Flow	1. User opens the Dashboard screen (default landing page). 2. System aggregates all transactions for the selected period. 3. System renders summary cards and charts. 4. User may change the period filter to refresh the view.
Postcondition	User has an up-to-date visual overview of their finances.

8. Software Testing Approach

8.1 Testing Levels

Level	Focus	Example
Unit Testing	Individual JS functions (e.g., balance calculation, validation).	calculateBalance() returns correct sum given a transaction array.
Integration Testing	Interaction between modules (e.g., transactions.js ↔ storage.js ↔ dashboard.js).	Adding a transaction correctly updates dashboard totals.
System Testing	End-to-end feature verification against functional requirements.	Full flow: add income → set budget → view report.
Usability Testing	Ease of use, layout clarity, responsiveness across devices.	Manual walkthrough on mobile and desktop viewports.

8.2 Sample Test Cases

Test ID	Description	Expected Result
TC-01	Add an expense of █500 in category "Food".	Balance decreases by 500; transaction appears in list and dashboard chart.
TC-02	Add a transaction with a negative amount.	System rejects input with a validation error message.
TC-03	Delete an existing transaction.	Transaction is removed from list; balance and charts update accordingly.
TC-04	Set a budget of █2000 for "Entertainment" and log █1800 in expenses.	Progress bar shows ~90% and a warning indicator is displayed.
TC-05	Reload the browser after adding transactions.	Previously entered transactions persist and reload correctly from localStorage.
TC-06	Resize browser window to mobile width.	Layout reflows responsively without broken elements or horizontal scroll.

8.3 Acceptance Criteria

The system will be considered acceptable for evaluation when all High-priority functional requirements (Section 3) are implemented and pass their corresponding test cases, the dashboard accurately reflects transaction data at all times, and the UI is responsive across desktop and mobile screen widths.

9. Other Requirements

9.1 Assumptions and Dependencies

- The evaluation will be conducted on a single device/browser; cross-device sync is out of scope for the current phase.
- The project assumes availability of a code editor and a modern browser for development and demonstration.
- Any charting library used (e.g., Chart.js) is assumed to be freely available via CDN or npm.

9.2 Future Scope

- Migrate persistence from localStorage to a full backend (Node.js/Express + database) using the API design in Section 6.4.
- Introduce multi-user authentication and cloud-based data sync across devices.
- Add recurring transactions (e.g., automatic monthly salary/rent entries).
- Add multi-currency support with live exchange rate conversion.
- Introduce AI-assisted spending insights and personalized savings recommendations.
- Add data import from bank statement files (CSV) for faster onboarding.

9.3 Risks and Mitigations

Risk	Mitigation
Data loss if browser storage is cleared.	Provide an Export (CSV/JSON) feature so users can back up data manually.
Scope creep beyond a realistic student timeline.	Prioritized requirements (High/Medium/Low) keep the MVP achievable; extras are Future Scope.
Inconsistent UI across screen sizes.	Use responsive CSS (Flexbox/Grid + media queries) and test on multiple breakpoints early.

9.4 Document Approval

Role	Name	Signature / Date
Project Team Lead	Saniya Jaiswal (NC)	_____
Project Guide / Faculty	_____	_____